

Práticas de Laboratório – 05 – Desenvolvimento de uma aplicação completa com a base da Persistência Poliglota.

Assunto: Desenvolvimento de uma Aplicação Streamlit com Persistência Poliglota: **Análise de Dados da OpenF1 com MongoDB e SQLite.**

Objetivo:

Projetar e desenvolver uma aplicação em Python utilizando o framework Streamlit que demonstre o conceito de **Persistência Poliglota**. A aplicação irá interagir com um banco de dados **MongoDB** contendo dados detalhados de sessões de Fórmula 1 (conforme o documento "Prática 04 - Manipulação dos Dados em Banco MongoDB") e, adicionalmente, irá gravar dados agregados ou sumarizados em um banco de dados relacional **SQLite**.

Contexto:

A aplicação, intitulada "OpenF1 Data Explorer", serve como uma interface para explorar dados de corrida armazenados em uma coleção MongoDB `openf1_data`. Atualmente, a ferramenta permite a visualização e análise comparativa do desempenho de pilotos por meio de filtros de sessão e gráficos interativos. O desafio é estender essa funcionalidade para incorporar um segundo banco de dados, aplicando o princípio da persistência poliglota.

Utilizaremos o **MongoDB** por sua flexibilidade e escalabilidade no armazenamento de dados de telemetria e voltas, que são numerosos e semiestruturados. Em paralelo, empregaremos o **SQLite** para armazenar de forma persistente os resultados de análises específicas, como relatórios de desempenho ou *insights* gerados pelo usuário, aproveitando a estrutura e a eficiência de um banco de dados relacional para dados mais estruturados e agregados.

Requisitos da Aplicação:

1. Estrutura do Projeto:

- **streamlit_app.py**: Arquivo principal da interface do usuário com Streamlit.
- **db_utils.py**: Módulo para gerenciar as conexões e consultas com MongoDB e SQLite.
- **requirements.txt**: Arquivo com as dependências do projeto (e.g., streamlit, pymongo, pandas, sqlalchemy).

- **.env**: Arquivo para armazenar a string de conexão do MongoDB de forma segura.

2. Funcionalidades de Leitura (MongoDB):

- Conectar-se de forma segura ao banco de dados MongoDB openf1_data.
- Permitir que o usuário selecione um ano e uma sessão de corrida específica (ex: Grande Prêmio da Itália 2023), populando os filtros a partir da coleção sessions.
- Exibir detalhes da sessão selecionada, como país e nome do circuito.
- Permitir a seleção de múltiplos pilotos daquela sessão para análise.
- Gerar um gráfico de linhas interativo comparando a duração de cada volta (lap_duration) pelo número da volta (lap_number) para os pilotos selecionados.
- Exibir uma tabela com os dados brutos das voltas dos pilotos selecionados.

3. Funcionalidades de Escrita e Leitura (SQLite - Persistência Poliglota):

- **Análise e Geração de Resumo:** Após a seleção dos pilotos e a visualização do gráfico, o usuário terá um botão "Gerar Resumo da Análise".
- **Cálculo de Métricas:** Ao clicar no botão, a aplicação deverá calcular métricas agregadas para os pilotos selecionados, como:
 - Volta mais rápida (lap_duration mínima).
 - Tempo médio de volta.
 - Número total de voltas completadas.
 - Consistência (desvio padrão dos tempos de volta, excluindo voltas de entrada e saída dos boxes).
- **Persistência em SQLite:**
 - Criar (se não existir) um banco de dados SQLite chamado analysis_reports.db.
 - Dentro deste banco, criar uma tabela race_analysis com colunas como session_name, driver_name, fastest_lap,

average_lap_time, total_laps, consistency_std_dev, e analysis_timestamp.

- Salvar as métricas calculadas na tabela race_analysis. Isso demonstra o uso do banco relacional para armazenar dados estruturados e sumarizados, que são o resultado de uma análise sobre os dados brutos do NoSQL.

- **Visualização dos Relatórios Salvos:**

- Implementar uma seção na aplicação chamada "Histórico de Análises".
- Nesta seção, a aplicação deve ler os dados da tabela race_analysis do SQLite e exibi-los em um st.dataframe, permitindo que os usuários vejam os resultados de análises anteriores de forma persistente.

Fluxo do Usuário:

1. O usuário abre a aplicação "OpenF1 Data Explorer".
2. Seleciona o ano "2023" e a corrida "Italian Grand Prix".
3. Escolhe os pilotos que deseja comparar (ex: Charles Leclerc e Carlos Sainz Jr.).
4. A aplicação busca os dados no MongoDB e exibe o gráfico de desempenho e a tabela de dados brutos.
5. O usuário clica no botão "Gerar Resumo da Análise".
6. O sistema calcula as métricas de desempenho e as salva na tabela race_analysis no arquivo analysis_reports.db (SQLite).
7. O usuário navega até a seção "Histórico de Análises" para visualizar uma tabela com todos os resumos de análises já salvos, lidos diretamente do SQLite.

Este prompt guiará o desenvolvimento de uma aplicação que não apenas visualiza dados de uma fonte NoSQL, mas também exemplifica a **persistência poliglota** ao utilizar um banco de dados SQL para armazenar de forma estruturada e duradoura os *insights* e agregações gerados a partir desses dados, aproveitando a melhor ferramenta para cada tipo de tarefa de armazenamento de dados.